PRAXIS ENGINEERING MANUAL

The Production Engineering & Industry Foundations Handbook

A Practitioner's Guide to Corporate Software Engineering, Culture, and Operations

Executive Overview: Theoretical computer science curricula teach data structures, algorithms, and toy programming assignments in isolated environments where code is written once and submitted for grading. In contrast, corporate production engineering takes place within living distributed systems where software must handle continuous traffic, concurrent deployments, unexpected outages, strict regulatory compliance, and multi-team collaboration.

This handbook bridges the gap between university theory and day-one workplace execution. As an engineer entering Praxis, you will learn the mental models, vocabulary, standards, and operational rituals required to ship reliable code, handle on-call responsibilities, collaborate across disciplines, and safeguard customer data.

> ***The Praxis Core Tenet:*** *In production, code clarity, observability, automated verification, and safe rollback mechanisms matter far more than clever or unreadable hacks. A bug in staging is a routine learning event; an unmitigated error in production damages customer trust and brand equity.*

Handbook Structure & Progression:

Section 1: Organizational Topology	Engineering structures, RACI matrices, and functional cross-collaboration.
Section 2: The Agile Lifecycle	Sprints, backlog refinement, user story estimation, and ticket states.
Section 3: Professional Version Control	Trunk-based development, semantic commits, and pull request hygiene.
Section 4: Environment Tiers & Isolation	Dev, QA, Staging, Ephemeral, and Production boundaries.
Section 5: CI/CD & Safe Deployments	Automated gates, Canary releases, Blue/Green, and feature flagging.
Section 6: Observability (Logs, Metrics, Traces)	Telemetry instrumentation, distributed tracing, and p99 profiling.
Section 7: SRE, SLOs & Incident Response	SLAs, error budgets, SEV categorization, and blameless postmortems.
Section 8: Enterprise Security & Compliance	Least Privilege, Zero Trust, OWASP principles, and data privacy.
Sections 9-10: Essential Industry Lexicon	40+ mission-critical architectural and operational acronyms decoded.

Section 11: The First 90 Days Playbook	Concrete habits for unblocking yourself and thriving in engineering teams.

01. Modern Engineering Organization Topology

Software in high-growth companies is built by cross-functional teams rather than isolated individuals. Understanding who owns which decisions prevents friction, accelerates feature delivery, and ensures accountability.

Engineering Manager (EM)	People manager focused on career growth, team capacity, sprint velocity, unblocking bottlenecks, and ensuring psychological safety.
Product Manager (PM)	Owns the 'Why' and 'What'. Discovers customer needs, writes Product Requirement Documents (PRDs), defines acceptance criteria, and manages feature prioritization.
Product Designer (UI/UX)	Designs user flows, wireframes, component design systems (e.g., Figma), conducts usability testing, and verifies accessibility (a11y).
Tech Lead / Staff Engineer	Owns the 'How' and long-term technical architecture. Authors Architecture Decision Records (ADRs), guides senior engineers, and guards system scalability.
Software Engineer (SWE)	Writes modular, tested, clean application code, authors pull requests, designs database schemas, and participates in code reviews.
Site Reliability Engineer (SRE)	Ensures availability, latency, performance, monitoring, and capacity planning. Builds automated infrastructure and manages deployment pipelines.
Quality Assurance / SDET	Automates regression, integration, and performance load tests, validating that releases meet critical business standards.
Security Operations (SecOps)	Performs vulnerability scans, static/dynamic code analysis (SAST/DAST), code audits, secret rotation, and incident triage.

The RACI Framework in Production Decisions

Engineering teams use the RACI matrix to clarify roles for key production deliverables: **R (Responsible)** = The engineer implementing the change; **A (Accountable)** = The lead who approves and signs off; **C (Consulted)** = SMEs (DBA, Security, SRE) who provide input; **I (Informed)** = Stakeholders notified upon rollout.

> ***Rule of Thumb:*** *Never deploy architectural changes or introduce third-party libraries without consulting your Tech Lead and Security team. Documenting decisions in an ADR saves hundreds of hours of rework.*

02. The Agile Delivery Engine & Work Tracking

Modern engineering teams work in 1- to 2-week iterations known as **Sprints**. Work is organized into backlogs and tracked using tools like Jira, Linear, or GitHub Projects.

Core Scrum & Kanban Ceremonies

- **Sprint Planning:** At the beginning of each cycle, the team reviews the backlog, breaks user stories into engineering tasks, and commits to a specific deliverable scope.
- **Daily Standup (15 mins):** Engineers provide three concise updates: (1) What was completed yesterday, (2) What is planned for today, and (3) Any blockers preventing forward progress.
- **Backlog Refinement / Grooming:** Engineers and PMs review upcoming feature proposals, clarify ambiguous acceptance criteria, identify dependencies, and assign story points.
- **Sprint Retrospective:** A blameless review at sprint conclusion addressing: What went well? What slowed us down? What concrete action items will we implement next cycle?

Story Pointing & Estimation (Fibonacci Sequence)

Story points measure relative complexity and uncertainty, not literal calendar hours. Common scale: **1 point** (trivial change, e.g. text copy or config bump), **2 points** (routine task with clear patterns), **3 points** (moderate feature with tests), **5 points** (significant logic spanning multiple components), **8+ points** (too large; must be decomposed into smaller sub-tasks).

Ticket Lifecycle & Status Progression

Every production task progresses through a standard workflow: **Backlog → In Progress → Pull Request / Code Review → Staging QA → Ready for Deploy → Production Verified → Closed**. Moving a ticket prematurely before tests pass or validation completes creates tracking chaos.

> ***Pro-Tip on Standups:*** *Never say 'I worked on tickets.' State your specific technical goal, such as: 'Yesterday I finished the Redis token cache migration; today I am writing integration tests for the rate limiter. I need a review from @Alex on PR #214.'*

03. Version Control Hygiene & Pull Request Etiquette

In production teams, Git is not just backup storage; it is the official audit trail of why, when, and how your software evolved.

Trunk-Based Development vs. Long-Lived Branches

High-performing teams practice **Trunk-Based Development**: engineers create short-lived feature branches (lasting hours to 2 days maximum), rebase frequently onto main, and merge often via automated CI checks. Long-lived feature branches lead to painful merge conflicts ('merge hell') and risky, high-blast-radius releases.

Conventional Commit Standards

Every commit should follow the Conventional Commits specification:

- **feat(auth):** Adds a new user-facing capability (e.g. feat(billing): add stripe webhook handler).
- **fix(database):** Patches an unexpected bug or error (e.g. fix(db): handle connection pool timeout).
- **refactor(api):** Modifies code structure without altering external behavior or adding features.
- **test(gateway):** Adds missing unit or integration tests without changing application logic.
- **chore(deps):** Routine maintenance, dependency upgrades, or CI config changes.

The Anatomy of a World-Class Pull Request

A junior engineer submits a PR with the title 'fixes stuff' and no description. A senior engineer provides complete context: (1) **Problem & Context:** What Jira ticket does this resolve? (2) **Solution Approach:** High-level architecture of changes, (3) **Verification & Testing:** CLI output or screenshots proving it works, and (4) **Rollback Plan:** What flag or command undoes this if an alert fires in production?

> ***Code Review Golden Rule:*** *Critique the code, never the person. Use prefixes like 'Nit: (optional style tweak)', 'Question: help me understand the concurrency model here', or 'Blocking: this query lacks an index and will table-scan in production.'*

04. Environment Tiers & Infrastructure Parity

Code progresses through distinct, isolated runtime environments before reaching paying users. Never test or debug experimental logic in production.

- **Localhost / Dev Container:** Developer workstation running local servers, mock databases, and unit test suites. Fast feedback loop, zero risk to external data.
- **Ephemeral / Preview Env:** On-demand environments spun up automatically for each pull request (e.g. Vercel preview or Kubernetes namespace). Enables PMs and designers to test the branch before merge.
- **Staging / UAT (User Acceptance):** A near-replica of production architecture with sanitised test data, external sandbox APIs (e.g. Stripe test keys), and realistic network latencies. Final gate for regression testing.
- **Production (Prod):** Live environment serving real customer traffic and real financial transactions. Strictly locked down; access requires multi-factor authentication (MFA) and audit logging.

The Twelve-Factor App: Config & Environment Parity

Never hardcode environment-specific URLs, API keys, or credentials into source code. Store configuration in environment variables injected at deployment time. Code that runs in Staging should be identical (same Docker container image tag) to code deployed to Production; only the injected environment variables differ.

Secrets Management Anti-Patterns

Accidentally committing an AWS key, JWT secret, or database password to Git is an immediate security incident requiring key revocation. Production secrets must be managed via encrypted vaults (AWS Secrets Manager, HashiCorp Vault, Doppler, or GCP Secret Manager), never stored in plaintext .env files tracked in Git.

> ***Blast Radius Principle:*** *Design systems so that failures are quarantined. A crash in the notification worker service must never take down the primary authentication or checkout API.*

05. CI/CD Pipelines & Modern Deployment Strategies

High-performing engineering teams deploy multiple times per day using automated delivery pipelines that eliminate manual human error.

Continuous Integration vs. Continuous Delivery vs. Continuous Deployment

- **Continuous Integration (CI):** Automated pipeline that triggers on every commit: runs linters (Ruff, ESLint), static type checkers (mypy, TypeScript), unit test suites, and vulnerability scanners.
- **Continuous Delivery (CD):** Automated generation of verified build artifacts (Docker images, binaries) and automated release to staging, awaiting manual approval before production rollout.
- **Continuous Deployment:** Every commit that passes all automated test gates is deployed directly to production without manual intervention. Requires mature observability and automated canary analysis.

Zero-Downtime Deployment Patterns

- **Rolling Update:** Containers are incrementally replaced one-by-one. The load balancer stops routing to old instances as new healthy instances pass health check probes.
- **Blue/Green Deployment:** Two identical production environments exist. 'Blue' serves live traffic. You deploy new version to 'Green'. Once verified, the load balancer router instantly flips traffic from Blue to Green.
- **Canary Release:** Route 2% of live traffic to the new version while 98% remains on stable. Monitor error rates, memory usage, and latency for 15 minutes. If metrics remain healthy, ramp to 10%, 50%, and 100%.
- **Feature Flags (Dark Launching):** Code is merged and deployed to production with functionality hidden behind a dynamic toggle (e.g. LaunchDarkly). Allows testing in prod with internal users before turning on globally.

> ***SOP Standard:*** *Always decouple deployment from feature release. Deploying code (shipping the bytes) should be a low-risk non-event; releasing features to users is controlled independently via feature toggles.*

06. Production Observability: Telemetry in the Wild

You cannot fix what you cannot measure. When an incident occurs at 2:00 AM, observability is the only lens through which engineers understand internal system state.

The Three Pillars of Observability

1. Metrics (Aggregated Numerics)
2. Structured Logs (Events & Context)
3. Distributed Tracing (Request Journeys)

Why Average Latency is a Trap: p50 vs. p95 vs. p99

Never report 'average latency is 40ms'. Averages hide catastrophic outliers. If 99 requests take 10ms but 1 request stalls for 10,000ms, the average is 109ms (which looks acceptable), but 1% of your highest-value customers experienced a total freeze. Production teams monitor **p95** and **p99 percentiles** (the maximum latency experienced by 95% and 99% of requests).

The Four Golden Signals (Google SRE)

- **Latency:** The time taken to service a request, segmented by successful vs. failed requests.
- **Traffic:** Demand placed on the system (e.g. HTTP requests per second or network I/O).
- **Errors:** Rate of failed requests (e.g. HTTP 5xx responses or uncaught exceptions).
- **Saturation:** How full the service is (e.g. CPU utilization, DB pool connection exhaustion).

07. Reliability Engineering, SLAs & Incident Triage

No distributed system achieves 100% uptime. SRE is the discipline of treating operational reliability as a software problem with mathematically defined boundaries.

SLI, SLO, SLA & Error Budgets

- **Service Level Indicator (SLI):** A quantifiable metric of service performance (e.g. '% of HTTP requests returning < 200ms without 5xx errors').
- **Service Level Objective (SLO):** The target reliability goal agreed upon by engineering and product (e.g. '99.9% of requests succeed over a rolling 30-day window').
- **Service Level Agreement (SLA):** The formal legal contract with customers specifying financial penalties or credits if the service breaches reliability thresholds (e.g. 99.5%).
- **Error Budget:** The permissible downtime or failure rate (100% - SLO). For a 99.9% SLO, you have an error budget of 0.1% (approx. 43 minutes of downtime per month). When the error budget is exhausted, feature deployments freeze and all focus shifts to reliability bugs.

Incident Severity Levels & On-Call Protocol

- **SEV-1 (Critical Outage):** Complete core system failure affecting all users (e.g. payment processing down). Requires immediate all-hands war room, updates every 15 minutes to executive leadership.
- **SEV-2 (Major Degradation):** High-impact issue affecting a substantial subset of customers with no immediate workaround (e.g. mobile app search returning 500 errors).
- **SEV-3 (Minor Incident):** Non-critical feature broken or workaround available (e.g. export CSV failing for certain accounts). Handled during normal business hours.

The Blameless Postmortem: *When things break, never ask 'Who caused this?' Ask 'What systemic failure allowed a human error to bypass our safeguards?' Human error is never the root cause; lack of automated validation, poor runbooks, or missing rollback safety is.*

08. Enterprise Security, Zero Trust & Compliance

Security is not a department; it is a fundamental property of every line of code you commit. Modern architectures assume the network is already hostile.

Fundamental Security Principles

- **Principle of Least Privilege (PoLP):** A service, database user, or human engineer should possess only the absolute minimum permissions necessary to perform their specific job—nothing more. Avoid granting admin or root roles.
- **Zero Trust Architecture:** Never trust internal network boundaries. Authenticate and authorize every single inter-service request using mutual TLS (mTLS) and cryptographically signed tokens (JWTs).
- **Defense in Depth:** Layer multiple defense mechanisms: WAF rate limiting at the edge, input sanitization in the API layer, parameterized queries at the ORM layer, and disk encryption at rest.
- **Sanitize All Inputs (OWASP Top 10):** Never concatenate strings into SQL queries. Always use parameterized queries or trusted ORMs. Sanitize user-generated content to prevent Cross-Site Scripting (XSS).

Personally Identifiable Information (PII) & Compliance

Regulatory standards such as GDPR, HIPAA, and SOC 2 mandate strict treatment of customer data. Never log passwords, credit card numbers, social security numbers, or sensitive medical information in application logs. Ensure databases support customer data deletion requests ('Right to be Forgotten').

> ***Security Golden Rule:*** *If you discover a security vulnerability or active exploit in production, do not discuss it in public Slack channels. Immediately escalate to your company's dedicated incident response team (e.g. #sec-ops-private) according to your security handbook.*

09. The Essential Industry Lexicon (Architectural)

Every production team uses domain jargon. Master these 20 architectural and infrastructure terms to navigate technical design reviews with confidence.

Idempotency	Property where an operation can be applied multiple times without changing the result beyond the initial application (crucial for payment APIs and webhook retries).
Circuit Breaker	Design pattern that halts requests to a failing downstream service after a threshold of errors, preventing cascading resource exhaustion across the cluster.
Eventual Consistency	Database consistency model where all replicas will eventually yield the same value, but immediate reads right after write may return slightly stale data.
Dead Letter Queue (DLQ)	A secondary queue holding messages that failed processing after maximum retry attempts, allowing manual inspection and replay without losing data.
Backpressure	Mechanism where a downstream consumer signals an upstream producer to slow down message rate when internal buffers are nearing capacity.
CDN (Content Delivery Network)	Geographically distributed network of edge proxy servers caching static assets (images, JS) near end-users to reduce origin server load.
WAF (Web Application Firewall)	Edge security filter protecting APIs against common web exploits such as SQL injection, cross-site scripting, and bot scraping floods.
Rate Limiting & Throttling	Restricting the number of requests a client can make in a given timeframe (e.g., Token Bucket algorithm allowing max 100 req/min).
Database Connection Pooling	Maintaining a cache of open database connections (e.g., HikariCP, PgBouncer) so queries reuse existing connections rather than paying high TCP handshake costs.
Write-Ahead Logging (WAL)	Technique where database modifications are recorded to append-only disk logs before writing to data files, guaranteeing ACID durability upon crash.

10. The Essential Industry Lexicon (Operations & Culture)

Engineering culture has its own distinct operational vernacular. Understanding these terms will help you decode everyday Slack conversations, tickets, and team retrospectives.

`Technical Debt ('Tech Debt')`	The implied cost of additional rework caused by choosing an easy or shortcut solution now instead of a better, scalable approach.
`Yak Shaving`	A chain of seemingly unrelated, tedious micro-tasks you must complete before you can make progress on your original engineering goal.
`Bike-Shedding`	The tendency of teams to spend disproportionate time debating minor, trivial details (like button color) while ignoring difficult, vital architectural issues.
`Bus Factor`	The minimum number of team members that, if suddenly unavailable (e.g. hit by a bus), would stall the project due to undocumented institutional knowledge.
`Runbook / SOP`	A step-by-step Standard Operating Procedure document detailing exact commands and steps to diagnose, mitigate, and resolve common production failure modes.
`MTTR & MTTD`	**Mean Time to Detect (MTTD):** Time between incident onset and alert firing. **Mean Time to Resolve (MTTR):** Time from alert to full system restoration.
`RCA (Root Cause Analysis)`	Formal investigation determining the foundational systemic vulnerabilities that triggered an incident, leading to preventive action items.
`Feature Freeze`	A temporary policy period (e.g. Black Friday or holiday peak) where no non-critical code changes are deployed to minimize risk of outages.
`Dark Launch`	Deploying new backend logic to production that handles real shadow traffic silently in the background before exposing the UI to users.
`Graceful Degradation`	System architecture that continues operating with limited functionality when components fail (e.g. Netflix showing default recommendations if ML service dies).

11. The First 90 Days Survival & Acceleration Guide

Transitioning from academic coursework to corporate production engineering can feel overwhelming. Follow these five high-leverage principles to establish credibility quickly:

The 30-Minute Rule for Being Stuck: When blocked on an issue: spend 30 minutes researching logs, searching the codebase, and reading internal documentation. If still stuck after 30 minutes, ask for help! When asking, document: (1) What you are trying to accomplish, (2) What you tried, (3) The exact error message, and (4) Your current hypothesis.

Improve Documentation as You Onboard: As a newcomer, you have fresh eyes. When you encounter out-of-date README steps or missing local setup scripts, submit a pull request fixing them immediately. Your teammates will appreciate it.

Read Code 3x More Than You Write: Before implementing a new feature or endpoint, inspect how three existing endpoints in the codebase handle error validation, database transactions, and logging. Follow established project conventions.

Ship Small, High-Frequency Pull Requests: Avoid working in isolation for two weeks and dumping a 2,000-line PR on your teammates. Break tasks into clean, logical PRs of 150-300 lines. They get reviewed in minutes and carry much lower deployment risk.

Own Your Outages and Mistakes: Every senior engineer has broken production at least once. If your PR triggers an alert, don't hide it or make excuses. Acknowledge it immediately in the incident channel, trigger a rollback, and assist in authoring the blameless postmortem.

> ***Welcome to Praxis:*** *You are now ready to take your Industry Readiness Assessment (5 MEQ Evaluation Questions) and begin tackling real production engineering missions. Strive for excellence, embrace blameless learning, and build systems that stand the test of scale.*